Gene Regulatory Dynamics

Joshua S. Weitz[1, 2, *]

[1] *School of Biology, Georgia Institute of Technology, Atlanta, GA, USA*
[2] *School of Physics, Georgia Institute of Technology, Atlanta, GA, USA*

(Dated: September 4, 2020)

I. CONTINUOUS MODELS OF CELLULAR DYNAMICS AND GENE REGULATION

The goal of this lab is to simulate the continuous dynamics of living systems. The technical details transcend particular scales, but having a focal example will help structure the effort and provide a basis for adapting lessons learned to different contexts, i.e., across molecules, cells, organisms, and ecosystems. Here, the focus is on gene regulation, i.e., how cells modulate the expression of their genes. Modulating gene expression ensures that not all genes are turned on at once. The core components involve the *transcription* of mRNA from DNA, the subsequent *translation* of mRNA into individual amino acids, which then fold into mature proteins. These individual events can be represented as processes that modulate the densities of both mRNA and proteins, i.e., the number of molecules per unit volume. As reference, the volume of *Escherichia coli* is approximately 1 μm^3, such that if there is one molecule in a volume of an *E. coli* cell, then that corresponds to a concentration of 1 nM (nanomole). This lab will use units of nM as the basis for analysis.

In order to simulate the dynamics of mRNA and proteins requires some simplification of the underlying processes. The figure below shows the core molecular players, described in detail in the main text. Gene regulation denotes the process by which transcription and translation are modified by the state of the system, i.e., by the densities of proteins themselves. This is a form of feedback, where the system state acts to modify process rates. Yet, even this level of simplification includes more components and processes than is necessary to build initial models of gene regulation and, in turn, to illustrate core concepts in the simulation of continuous dynamical systems. Instead, the complex cast of players can be reduced into a two-component system involving the densities of mRNA (m) and protein (p). mRNA is transcribed at a rate potentially modified by transcription factors (themselves proteins), then mRNA is translated into proteins, and both mRNA and proteins decay (or are diluted). How does one move from this simplified representation to a representative set of equations and, in turn, a simulation of gene expression dynamics?

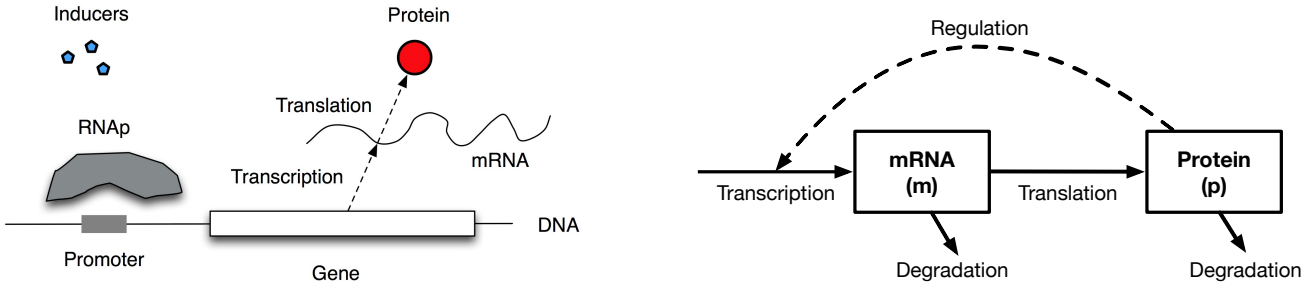


FIG. 1: Gene regulation in a nutshell. (Left) The molecular players and mechanisms involved in gene regulation. (Right) A schematic view of a model involving the dynamics of mRNA and protein abundances only.

To answer this question first requires a mathematical representation of the rules governing changes in concentrations. According to the central dogma of molecular biology, information encoded in genes within DNA is transcribed into mRNA which is then translated into proteins at the ribosome. Here, we consider the dynamics of a constituitively active promoter, that is, one that constantly recruit RNA polymerase and produces mRNA of the corresponding gene. By assuming an abundance of RNA polymerase, we can assume the mRNA produce protein copies at a constant per

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

capita rate. This leads to dynamics of mRNA concentration, m, and protein concentration, p:

$$\frac{\mathrm{dm}}{\mathrm{dt}} = \overbrace{a}^{\text{transcription}} - \overbrace{b_m m}^{\text{degradation}} \tag{1}$$

$$\frac{\mathrm{dp}}{\mathrm{dt}} = \overbrace{rm}^{\text{translation}} - \overbrace{b_p p}^{\text{degradation}} \tag{2}$$

where a is the rate of mRNA transcription and r is the rate of translation per transcript. The concentration of mRNA and proteins are reduced by dilution at rate b. This dilution occurs because m and p refer to concentrations (number/volume) and the volume of cell grows over time. In total, these represent a coupled system of ordinary differential equations. These equations happen to be linear, but the framework for analyzing them can apply to both linear and nonlinear equations. Compare these equations to Figure 1-right. Note that each of the solid arrows in Figure 1 corresponds to one term in the coupled system of differential equations. In general, care must be taken to ensure that every process is represented both by additions and losses to the abundances of the relevant systems (e.g. at population scales, predation involves both the loss of prey biomass and the gain of predator biomass). The dashed arrow denotes the fact that the rate of transcription might also depend on the levels of protein, i.e., $a \to a(p)$, where $a(p)$ denotes the fact that transcriptional rates depend on the concentrations of proteins, this dependency could be nonlinear.

Given a set of differential equations, how does one solve them? That is to say, given the initial transcript and protein levels in a cell, and rates of transcription, translation, and degradation, what is the expected values of $m(t)$ and $p(t)$, i.e., the time-course of mRNA and protein, respectively? Answering this equation requires the simulation and analysis of dynamical systems. Core methods for analyzing ordinary differential equations apply whether the underlying model represents gene regulation, neuronal dynamics, or predator-prey dynamics. Simulating this system on a computer requires some way of discretizing continuous time to update the state variables, i.e., in this case $m(t)$ and $p(t)$. This updating is called numerical integration; which can be performed using standard functions, i.e., `scipy.integrate.odeint` in Python. The core idea underlying these functions can be understood as follows. First, the instantaneous rate of change is calculated for all variables, i.e., dp/dt and dm/dt. Then, given a suitably small interval of time, the derivative is multiplied by a finite time interval, Δt to yield an incremental change in the state variables, i.e., Δm and Δp. Given the updated values, the process repeats (see Figure 2 for a schematic).

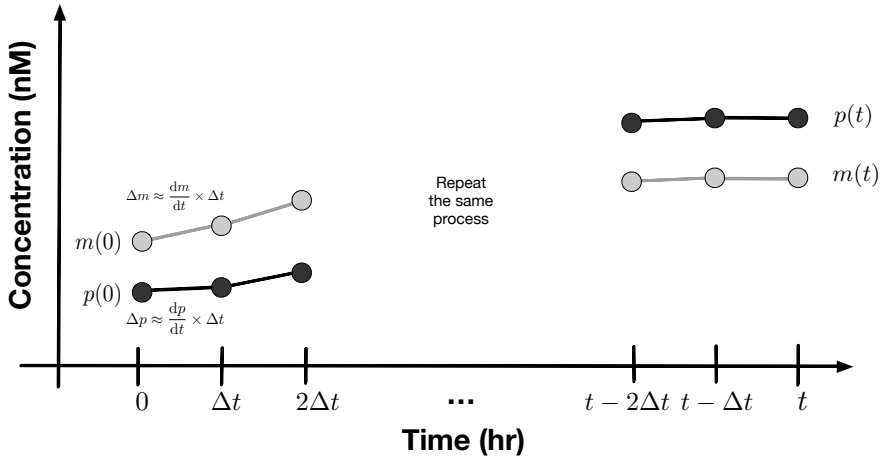


FIG. 2: Schematic of the Euler integration process.

One major point of caution: do not actually use a discrete implementation of the Euler integration process, it is numerically unstable and will almost certainly give erroneous results. In reality the function `scipy.integrate.odeint` in Python. uses a more sophisticated "higher-order" scheme to ensure that small errors do not compound to ever larger errors in the course of the integration.

But, numerical integration is only part of the story. This laboratory will introduce a varied toolkit to be deployed in the service of analyzing dynamical systems in the life sciences. The concepts and practical approaches include the identification of fixed points, nullclines, stability analysis, and even bifurcation analysis.

II. SIMULATING COUPLED ORDINARY DIFFERENTIAL EQUATIONS

As noted in the introductory remarks to this laboratory, solving equations, like those of mRNA and protein dynamics, can be difficult or impossible in more complex scenarios. However, numerically solving equations like these with a computer is relatively straightforward. Numerically integrating a coupled system of ODEs is performed via `scipy.integrate.odeint` in Python which is run with the following command:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import integrate
t = np.linspace(t0,tf)
x = integrate.odeint(odefun,x0,t,args=(pars,))
```

In this function `t` are points in time, `x` is the solution of the differential equation, `odefun` is the name of the function where you define the set of differential equations, `t0` is the initial time, `tf` is the final time, `x0` is the initial value of `x`, and `pars` is a structure that can be used to pass parameters into the function `odefun` It is important to note that the variable `x` can represent a vector of variables, e.g., both $m(t)$ and $p(t)$. The last input variable is optional. Hence, without passing any variables, then this program can be run as:

```
x = integrate.odeint(odefun,x0,t)
```

The differential equation for the protein model is represented with the function:

```
def proteinproduction(x,t,pars):
    """
    Returns the instantaneous rate of change given the current time t,
    state variable x, and parameters (pars)
    """
    m=x[0]
    p=x[1]
    dmdt=pars['a']-pars['bm']*m
    dpdt=pars['r']*m-pars['bp']*p
    dxdt = np.array([dmdt,dpdt])
    return(dxdt)
```

In this function, the set of variables embedded in `x` is converted into variable names that correspond to the m and p. This conversion is intentional to help make the code readable, even at a minor cost of some efficiency. Note, `dxdt` is an array with length equal to the number of variables in the system. In Python, the instantaneous rate of change is a vector, e.g., `dxdt=np.zeros(4)` would be a way to initialize the function in the case of a 4-dimensional dynamical system.

But this function is just the start – it describes the instantaneous rate of change, which itself must be used as part of a higher-order 'Runga-Kutta' method to calculate the trajectory. The dynamics can be simulated as follows:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import integrate
from lab3_functions import proteinproduction

# Parameters
pars={} #initiate parameter dictionary
pars['a']=10 #nM/hr
pars['bm']=2 # /hr
pars['bp']=1 #/hr
pars['r']=30#proteins/(mRNA*hr)

# Initial conditions
x0=[0,0] #Initial values
t0 = 0 #Time to start
tf = 1 # final time
t=np.linspace(t0,tf)
```

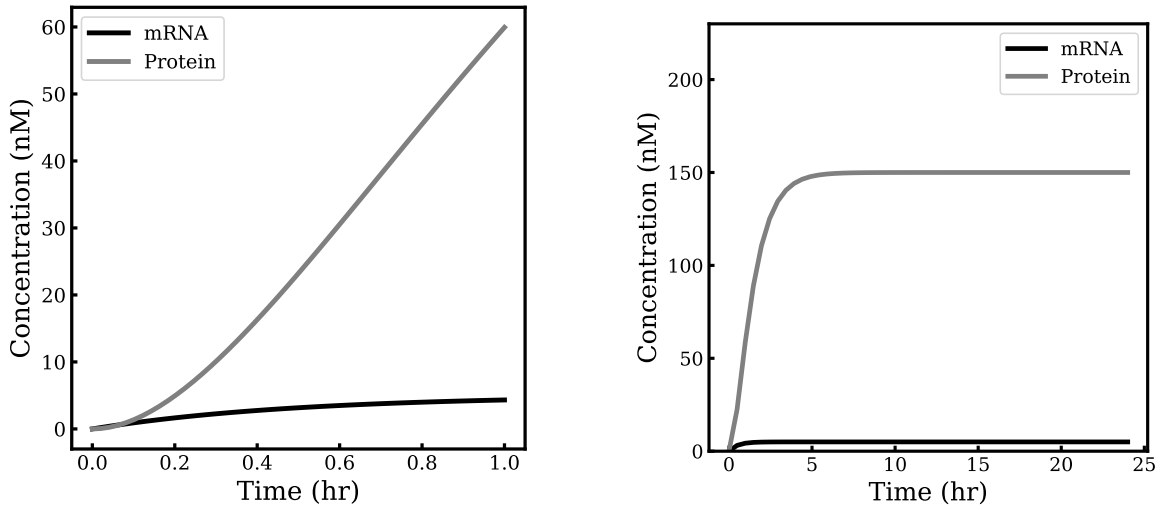
```
# Simulate
x=integrate.odeint(proteinproduction,x0,t,args=(pars,))

# Visualize
fig = plt.figure()
ax = fig.gca()

plt.plot(t,x[:,0],linewidth=3,color='k')
plt.plot(t,x[:,1],linewidth=3,color=[0.5,0.5,0.5])
plt.xlabel('Time (hr)',fontsize=20)
plt.ylabel('Concentration (nM)',fontsize=20)

plt.legend(['mRNA','Protein'],fontsize=20)
ax.tick_params(labelsize=20)
```

The result is a plot showing the growth of protein and mRNA over 1 hr. Note the units of time (x-axis) and concentration (y-axis). Next to it is the result of dynamics until the system reaches steady state (as a self-check, modify the code to simulate out to 24 hrs by modifying the final time).



This simulation manages to condense many biological parameters into a single variable – `pars`. There are many ways to organize variables, e.g., in Python, one way is to use 'dictionaries'. Organizing your variables is a convenient way to group multiple parameters together, both to help read and re-use code but also to facilitate passing information from scripts to functions. In Python, a dictionary is a variable that links keys to outputs. The code above introduced four, corresponding to the parameters: a, b_m, b_p, and r. It did so by using the `pars[key]` notation where the variable name is the key and the value is on the right hand side of the equal sign. In the code block above, these variables have a value, $a = 10$, $b_m = 2$, $b_p = 1$, and $r = 30$, respectively. However, fields can also denote strings, matrices, and even structures inside structures. Let's give it a try.

Challenge problem: Organizing variables

This challenge has two parts: (i) add a new field termed `aRange` to the dictionary `pars` with values from 10 to 50 with a step size of 2. (ii) create a new dictionary called `stats` with three matrices as fields, `tvals`, `mvals`, and `pvals`, each with the number of columns as the length of `aRange` and 100 rows. Initialize all entries to 0. Notably, this format can be used to collect results from many simulations in a single variable.

Solutions: Challenge problem: Organizing Variables
Challenge (i): add a new field termed `aRange` with values from 5 to 50 with a step size of 2.

```
# Parameters
pars={} #initiate parameter dictionary
pars['a']=10 #nM/hr
pars['bm']=2 # /hr
pars['bp']=1 #/hr
pars['r']=30#proteins/(mRNA*hr)
# New field-value
pars['aRange'] = np.arange(10,50,2)
```

Challenge (ii): create a new structure called `stats` with three matrices as fields, `tvals`, `mvals`, and `pvals`, each with the number of columns as the length of `arange` and 100 rows. Initialize all entries to 0.

```
stats = {}
stats['tvals'] = np.zeros( (100,len(pars['aRange'])) )
stats['pvals'] = np.zeros( (100,len(pars['aRange'])) )
stats['mvals'] = np.zeros( (100,len(pars['aRange'])) )
```

Note, you must initialize the dictionary with `dictName = {}`.

III. QUALITATIVE ANALYSIS OF NONLINEAR DYNAMICAL SYSTEMS

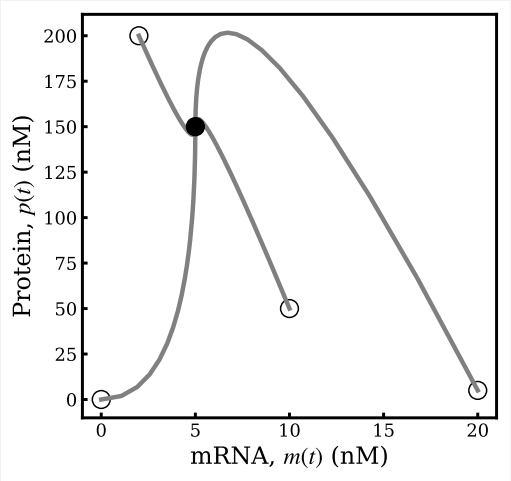
A. Phase plane visualizations

The previous simulations converged to the values $m(t) \to 5$ nM and $p(t) \to 150$. This was expected. In this case, the dynamics converged to a "fixed point", i.e., a combination of state variables in which there is no subsequent change. This fixed point occurs when dm/dt = 0 and dp/dt = 0. This is satisfied when $m^* = a/b_m$ and $p^* = \frac{rm^*}{b_p}$, or $m^* = 5$ nM and $p^* = 150$ nM. Yet what happens when the system is not at that fixed point?

To analyze system dynamics requires additional approaches, particularly focusing on transient dynamics. Steps in this direction require that we think differently about how dynamics change in time. Whereas most biologists tend to think about how each variable changes with respect to time, e.g., visualizing many time-series stacked one on top of another, an alternative approach is to think about how each variable changes with respect to each other. As such, the dynamics of coupled systems can be represented in terms of their dynamics in phase space, e.g., here the plane represented by $(m(t), p(t))$ where time is implicit rather than explicit. In general, the dimensionality of the phase space is as large as there are variables. Visualizing such dynamics is possible for low-dimensional systems, like the $m(t)$ and $p(t)$ system studied here.

Challenge problem: Visualizing Phase Space Dynamics

Write code that visualizes phase-space dynamics of a mRNA-protein system over a 24 hr period with parameters as above, given 4 different sets of initial conditions $(m(0), p(0)) = \{(0,0), (20,5), (2,200), (10,50)\}$. The resulting dynamics should look as shown in this box, where open circles denote the initial conditions, the gray curve denote trajectories, and the black closed circle denotes the expected fixed point:
Hint: use the command `plt.plot` to plot the two output components against one another, corresponding to $m(t)$ and $p(t)$, respectively. In addition, using an array of initial conditions can help with your overlay.



Solutions to Challenge Problem: Visualizing Phase Space Dynamics The solution to this problem involves overlaying multiple phase space dynamics on the same plot. The code snippet below also includes labeling commands.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import integrate
from lab3_functions import *

# Parameters
pars={} #initiate parameter dictionary
pars['a']=10 #nM/hr
pars['bm']=2 # /hr
pars['bp']=1 #/hr
pars['r']=30#proteins/(mRNA*hr)

# Initial conditions
#Array of initial values
x0_range=[[0,0],
          [20,5],
          [2,200],
          [10,50]]
t0 = 0 #Time to start
tf = 24 # final time
t=np.linspace(t0,tf)

# Simulate/visualize
for i in range(len(x0_range)):
    x0 = x0_range[i]
    plt.plot(x0[0],x0[1],
             marker='o',
             markersize=12,
             markeredgecolor='k',
             markerfacecolor='w')
    x = integrate.odeint(proteinproduction,x0,t,args=(pars,))
    plt.plot(x[:,0],x[:,1],color=[0.5,0.5,0.5],linewidth=3)

#Visualize the expected fixed point
plt.plot(pars['a']/pars['bm'],pars['r']*pars['a']/(pars['bm']*pars['bp']),
            marker='o',
            markersize=12,
            color='k')

#Label
plt.xlabel('mRNA, m(t) (nM)',fontsize=20)
plt.ylabel('Protein, p(t) (nM)',fontsize=20)
```

B. Nullclines

The next step in qualitative analysis is to understand the trajectory of a system in phase space even when far from the fixed point. To do so it will help to identify the "nullclines" of the system. A nullcline of a particular variables denotes the set of points in phase space where the respective dynamical equation is set to 0. Hence, nullclines are associated with each variable, e.g., there are m and p nullclines. For the mRNA-protein model, solving for the

nullclines leads to:

$$\frac{dm}{dt} = 0 \implies m = \frac{a}{b_m} \tag{3}$$

$$\frac{dp}{dt} = 0 \implies p = \frac{rm}{b_p} \tag{4}$$

In other words, there is no change in the mRNA concentration whenever $m(t) = a/b_m$. In addition, there is no change in protein concentration whenever $p(t) = rm(t)/b_p$, i.e. when the ratio of proteins to mRNA is equal to r/b_p. Identifying nullclines also helps to identify the sign of change of variables, e.g., there is usually a sign changed associated with the rate of change of one variable when moving one side of the associated nullcline to the other.

The following commands show how to plot these nullclines in the mRNA-protein space, including the use of anonymous functions:

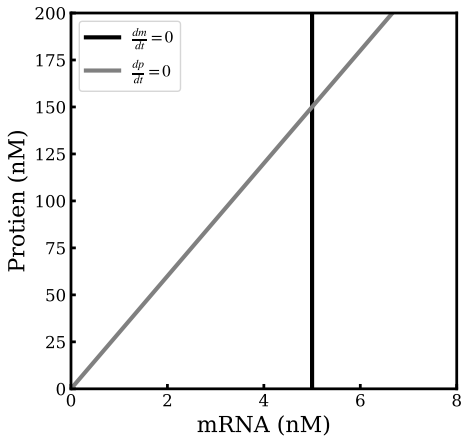
```
# Define the nullclines
mnullcline = pars['a']/pars['bm']    # A fixed value
pnullcline = lambda p : pars['bp']/pars['r'] * p     # An 'anonymous' function

# Define a vector of protein and mRNA values
pvec = np.arange(0,201,20)
mvec = np.arange(0,9)

# Plot the nullclines
plt.plot(mnullcline*np.ones(np.shape(pvec)), pvec, linewidth=3)
plt.plot(pnullcline(pvec), pvec, linewidth=3)

# Label the plot
plt.legend([r'$\frac{dm}{dt} = 0$',r'$\frac{dp}{dt} = 0$'],fontsize=18)
plt.xlim(min(mvec),max(mvec))
plt.ylim(min(pvec),max(pvec))
plt.xlabel('mRNA concentration (nMol)',fontsize=20)
plt.ylabel('Protien Concentration (nMol)',fontsize=20)
```

The variable `pnullcline` is an anonymous function. The `lambda` keyword means that the input argument 'p' in the assignment is a variable and must be provided as an input when calling `pnullcline`. Note that like any function, you don't need to call the variable provided to the function by the same name as the local function, and you can even pass in a fixed, scalar value. This code gives:

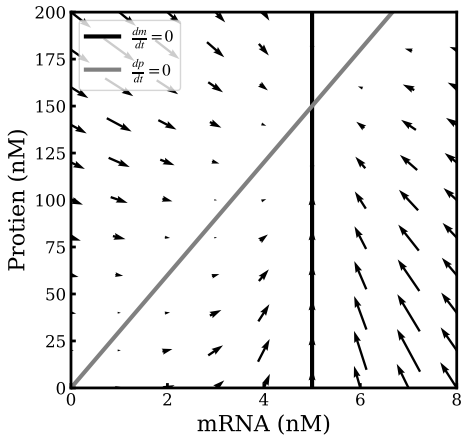


Note that the nullclines cross at precisely one location. At this location both dm/dt = 0 and dp/dt = 0. Hence, the intersection of the lines is the equilibrium of the system (m^*, p^*). To visualize the expected local dynamics, it is possible to visualize $\frac{dm}{dt}$ and $\frac{dp}{dt}$ throughout phase space. One way to do so is to plot a mesh of arrows that point in the local direction of change, as follows using the `plt.quiver` function in Python. Note that the following code already presumes the existence of the `mvec` and `pvec` variables as defined above.

```
# Define rate of change
dmdt = lambda pars,m: pars['a']-pars['bm']*m
dpdt = lambda pars,m,p : pars['r']*m-pars['bp']*p
[MM,PP] = np.meshgrid(mvec,pvec)

# Replot the nullclines
plt.plot(mnullcline*np.ones(np.shape(pvec)),pvec)
plt.plot(pnullcline(pvec),pvec)

# Overlay the arrows
plt.quiver(MM,PP,dmdt(pars,MM),dpdt(pars,MM,PP),pivot='mid',
           angles='xy')
```



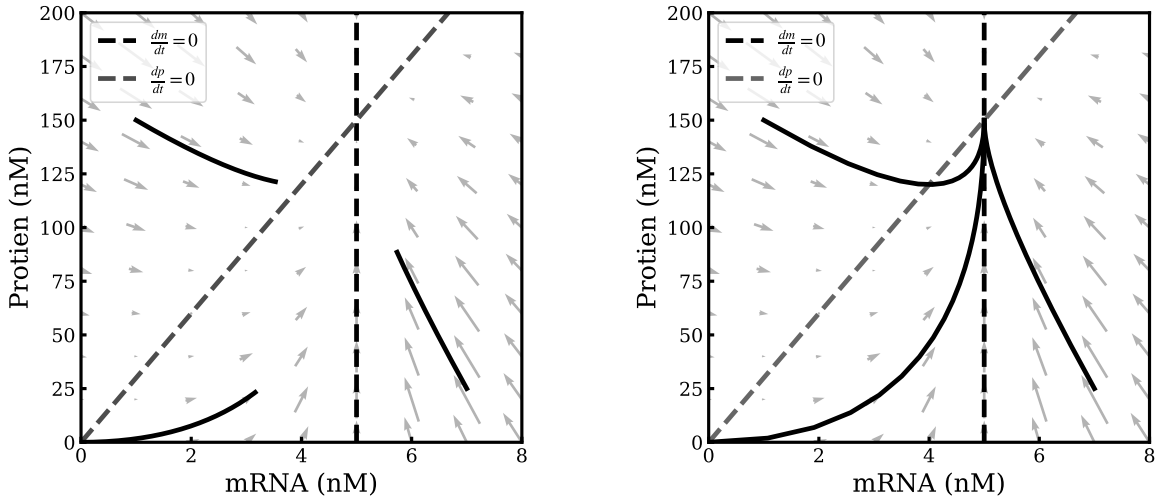
Of course the nullclines do not necessarily define the actual dynamics. First of all, dynamics can cross nullclines, because they only define where one of the variables (not two variables or all in higher-dimensional systems) have zero rate of change. Instead, the 'velocity field' depicts the local direction and magnitude of flow. This velocity field can be used to help interpret anticipated dynamics and even stability as examined in the next challenge problem.

Challenge Problem: Connecting Dynamics to Nullclines

Simulate Eqs.(1)-(2) beginning with three initial values in (m, p) space: $(0, 0)$, $(7, 25)$, and $(1, 150)$ over a 0.5 hr period. What do you expect to happen initially? Will both mRNA and protein increase in all cases? Where should the dynamics converge? Then, overlay the dynamics onto the velocity field for all three starting conditions, over a 24 hr simulation - to what extent did the null clines and 'quivers' help anticipate trajectories?

Solutions to Challenge Problem: Connecting Dynamics to Nullclines
The dynamics depend on the starting conditions. For both $(0, 0)$ and $(7, 25)$, the system converges to the equilibrium at $(5, 150)$. For $(1, 150)$, first proteins decrease and then recover as new mRNA is produced. The realized dynamics are shown are shown below, where the code assumes that you are adding these dynamics on top of the prior visualization commands using lighter arrows to highlight the vector fields and dashed lines to plot the nullclines.

```
m0_vec = [0, 1, 7]
p0_vec = [0, 150, 25]
t0=0
tf=24
t=np.linspace(t0,tf,200)
for i in range(3):
    x0=[m0_vec[i], p0_vec[i]]
    x = integrate.odeint(proteinproduction,x0,t,args=(pars,))
    plt.plot(x[:,0],x[:,1],linewidth=3,color='r')
#Don't forget to replot the nullclines and quivers
plt.plot(mnullcline*np.ones(np.shape(pvec)),pvec)
plt.plot(pnullcline(pvec), pvec)
plt.quiver(MM,PP,dmdt(pars,MM),dpdt(pars,MM,PP),pivot='mid',
                  angles='xy')
```



As is apparent, the dynamics have not yet reached equilibrium after half an hour (and appear to not even be that close). The dynamics all converge to the fixed point by 24 hrs, and as is apparent, the dynamics can and do cross the nullclines. The other take-away point is that the nullclines provide an indication of local change. Note also that the sign of either the $\dot{m}$ or $\dot{p}$ changes each time the dynamics cross a nullcline.

IV. EVALUATE THE LOCAL STABILITY OF EQUILIBRIA

Visually at least, the equilibrium appears to be locally stable, in the sense that trajectories approach it asymptotically. In general, there may be multiple locally stable equilibria in a system. Dynamics approach stable equilibria only if they exist within the associated "basin of attraction", i.e., a region in the phase space – see main notes for more information. The shape of these basins can be complicated and are not guaranteed to be of a certain size. Hence, to numerically address local stability it is worth first identifying equilibria either analytically by or numerically by solving for the intersection of all nullclines. Then, a full accounting of the local stability can usually be assessed through linearization of the dynamical systems near a fixed point. Numerical simulations via small perturbations can be used to verify analytic predictions of local stability or to provide preliminary numerical evidence in the absence of analytical results.

But first, we must identify the equilibria of the system. As was suggested above, the fixed points of a system $\mathrm{dx_i/dt} = \mathrm{f_i(x, \theta)}$ must satisfy $f_i(x^*, \theta) = 0$ for all x_i. Repeatedly solving for variables using the nullclines and

plugging into the solutions for the other nullclines solves the algebraic system. This yields an equation for equilibria. But, solving such equations is not always guaranteed. By construction, the present case is straightforward:

$$\frac{dm}{dt} = 0 \qquad \implies m^* = \tfrac{a}{b_m} \tag{5}$$

$$\frac{dp}{dt} = 0 \implies rm^* = bp^* \implies \tfrac{ra}{b_m} = b_p p^* \implies p^* = \tfrac{ra}{b_m b_p} \tag{6}$$

The present parameters yield $p^* = 150$ nM and $m^* = 5$ nM. Numerical solutions for equilibria can also be used in the event that there are multiple equilibria and solving analytically is difficult, e.g., via the `scipy.optimize.fsolve` command. One approach is to re-arrange the solution of the differential equation at equilibria and provide a guess of the value.

```
from scipy.optimize import fsolve
allnull = lambda p : pars['r']*pars['a']/pars['bm'] - pars['bp']*p
equil_num = fsolve(allnull,100)
```

Here we guess p^* is 100 at equilibria. By varying our guesses throughout parameter space it may be possible to identify potential multiple equilibria. More generally, nullclines are useful in identifying how many equilibria exist in the system.

A. Analytical formalism for linear stability analysis

The local stability of a (nonlinear) dynamical system can be assessed through a series of steps.

- An equilibrium is identified, which we will denote as x^*.
- The nonlinear dynamical system is linearized, in essence by approximating the full nonlinear dynamics of the system via a Taylor expansion near the fixed point. The linearized dynamics is described by the Jacobian (see textbook for full details).
- The stability of the system is identified by finding the eigenvectors and eigenvalues of the Jacobian. The eigenvectors denote those directions in phase space in which a perturbation will grow or shrink exponentially without changing its direction.

In the event that at least one eigenvalue is positive, this implies that small perturbations that have some 'projection' in the associated eigenvector direction will grow exponential. In essence, if one or more eigenvalues has a positive, real component, then the equilibrium, is unstable. If all eigenvalues have negative real components then the equilibrium is stable. We will return to the borderline case of zero eigenvalues later.

How can we turn these relatively small number, but potentially opaque, series of steps into practice? That is: how does one calculate eigenvalues and eigenvectors numerically? Recall that the Jacobian is the linearized version of dynamics near the fixed point. Let's refer to the small perturbations $u = m - m^*$ and $v = p - p^*$. Because the original system was already linear, this transformation yields a new system of equations:

$$\frac{du}{dt} = a - b_m(u + m^*) \tag{7}$$

$$\frac{dv}{dt} = r(u + m^*) - b_p(v + p^*) \tag{8}$$

which replacing the fixed point values yields:

$$\frac{du}{dt} = -b_m u \tag{9}$$

$$\frac{dv}{dt} = ru - b_p v. \tag{10}$$

This dynamical system is linear – and would be even if the original had been nonlinear if one ignores higher order terms in u and v. As such we expect the time-dependent solutions to be combinations of exponential, such that the derivatives with respect to time are just a constant rate factor multiplied by the state variable i.e., du/dt = λu and dv/dt = λv where λ is the unknown eigenvalue.

We can therefore rewrite these dynamics of perturbations as:

$$\lambda \begin{bmatrix} u \\ v \end{bmatrix} = J \begin{bmatrix} u \\ v \end{bmatrix} \tag{11}$$

where

$$J = \begin{bmatrix} -b_m & 0 \\ r & -b_p \end{bmatrix}. \tag{12}$$

Here J is the Jacobian matrix. Although the entries in this particular matrix are all fixed parameters, for nonlinear system these entries will also depend on the state variables. In that case, the Jacobian must be evaluated at the fixed point of relevance, i.e., substituting the values of each variable with its corresponding value. The eigenvalues are the solutions to the equation

$$\mathrm{Det}(J - \lambda I) \tag{13}$$

where λ is an eigenvalue and I is the 2×2 identity matrix, i.e.,

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{14}$$

There are as many eigenvalues as there are dimensions in phase space. Hence for the mRNA-protein system there are two eigenvalues. In this case, because of the absence of feedback between protein and mRNA, the two eigenvalues are $\lambda_1 = -b_m$ and $\lambda_2 = -b_p$. Because both are negative, the system must be locally stable. This result provides the basis for numerical verification in the next section.

B. Numerical simulations near equilibria

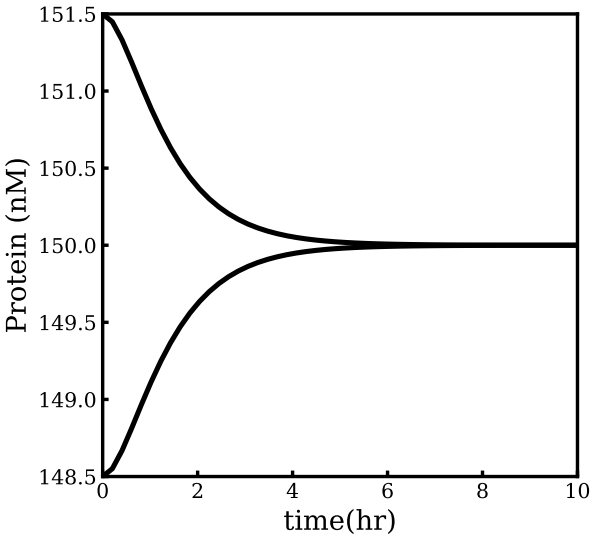
Numerically evaluating the stability of an equilibrium point is possible by simulating the system dynamics near to a fixed point. The following code snippet shows how to do so for the mRNA-protein case.

```
# Find fixed points
pstar=pars['r']*pars['a']/pars['bm']/pars['bp']
mstar=pars['a']/pars['bm']

# Perturb initial conditions
pperturb=pstar*0.01
mperturb = mstar*0.01
x0=[mstar+mperturb,pstar+pperturb]
t0=0
tf=10
t=np.linspace(t0,tf)

# Simulate and visualize just the proteins
x=integrate.odeint(proteinproduction,x0,t,args=(pars,))
plt.plot(t,x[:,1],linewidth=3,color='b')
plt.xlabel('time(hr)',fontsize=20)
plt.ylabel('Protein Concentration (nMol)',fontsize=20)
```

This code snippet perturbed the system 1% above the equilibria and only plotted the protein dynamics. A similar plot could show that mRNA also returns to the fixed point. By including a second condition just below the fixed point, the code should yield a plot that looks like this:

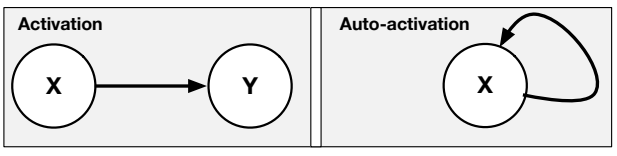


V. BISTABILITY AND BIFURCATION DIAGRAMS

This last section is meant to form the final bridge to the problem set in which you will attempt to recapitulate the findings underlying a synthetic toggle switch in *E. coli*. The toggle switch operates given a principle of 'bistability' – the idea that a system initialized in different parts of the phase space may end up in different states. These states are also locally stable, i.e., once they end up there, then the system will relax back locally given small perturbations. However, bistability does depend on parameters. Hence, as one changes parameters, then the value and even the number of equilibria of the system can change. When a system switches the number and/or stability of its equilibria, we can say the system has undergone a 'bifurcation'. These two ideas – bistability and bifurcations – can be assessed numerically. A full analysis will take time, and these concepts will recur throughout both this laboratory guide and the main text. Nonetheless, the following sections will help set you on the path to understanding and implementation in practice.

A. Simulating bistability in one dimension

Gene regulatory dynamics often involves feedback where the levels of proteins modulate the subsequent expression of genes in a network. By way of context, activation (or positive regulation) refers to the case when protein X binds upstream of gene Y and *increases* the production of mRNA and subsequently the protein Y. Similarly, inhibition (or negative regulation) refers to the case when protein X binds upstream of gene Y and *decreases* the production of mRNA and subsequently the protein Y. The toggle switch designed and synthesized by Gardner and colleagues involves two genes that mutually inhibit one another. This symmetric inhibition can lead to bistability. Hence, even if we were to ignore the mRNA dynamics, it would still require a two-dimensional analysis of the concentration of the two relevant proteins each of which function as transcription factors – a perfect topic for the main text and homework. Yet, the concept of bistability can be explored in one-dimension. To do so, consider the expression from a gene in which the protein acts as a transcription factor, i.e., modulating the expression of itself.



In a self-activating loop, the protein dynamics can be written in the following form:

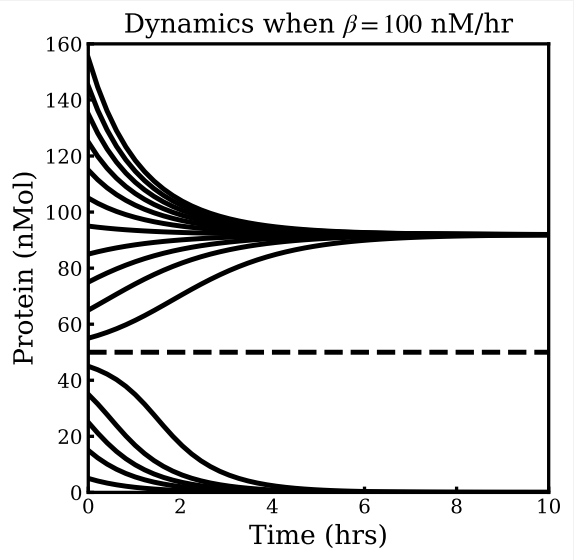
$$\frac{\mathrm{dx}}{\mathrm{dt}} = \frac{\beta x^n}{K^n + x^n} - \alpha x \tag{15}$$

where x is the density of proteins, β is the maximal production rate, K is the half-saturation constant, α is the decay/dilution rate, and n is a Hill coefficient corresponding to the degree of cooperativity for positive feedback. This model of positive feedback is sufficient, in certain conditions, to generate bistability. The next challenge problem allows you to start the process of exploring bistability on your own. The first step is to construct a function for the dynamical system, as in the following code snippet:

```
def positivefeedback(x,t,pars):
    """
    Returns the instantaneous rate of change given the current time t,
    state variable x, and parameters (pars)
    for a 1D positive feedback loop
    """
    dxdt = pars['beta']*x**pars['n']/(pars['K']**pars['n'] + x**pars['n'])-\
           pars['alpha']*x
    return(dxdt)
```

Challenge Problem: Simulating a bistable feedback loop

Write your own script to simulate a positive feedback loop and demonstrate that different initial conditions lead to different outcomes. Use the following parameters: $\beta = 100$ nM/hr, $K = 50$ nM, $n = 4$, $\alpha = 1$. If your code works, it should lead to a result like the following:



Solutions to Challenge Problem: Simulating a bistable feedback loop The following script can be used to simulate a positive feedback loop and demonstrate that different initial conditions lead to different outcomes:

```
# Parameters
pars={} #initiate parameter dictionary
pars['beta']=100
pars['K']=50
pars['n']=4
pars['alpha']=1

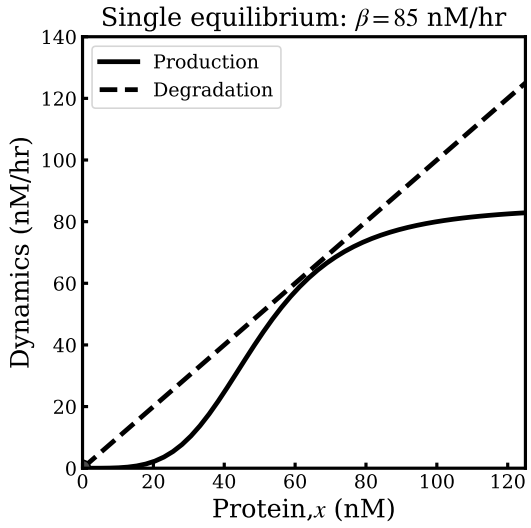
#Set time limits
t0=0
tf=10
t = np.linspace(t0,tf)
#Simulate
x0range = np.arange(5,165,10)
for i in range(len(x0range)):
    x0 = x0range[i]
    x = integrate.odeint(positivefeedback,x0,t,args=(pars,))
    plt.plot(t,x,color='k',linewidth=3)

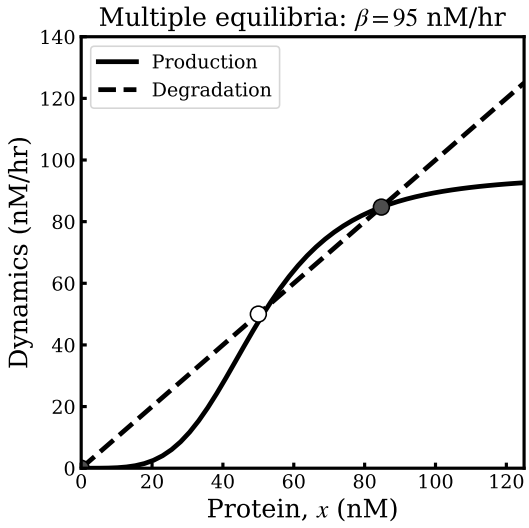
# Visualize the split point
plt.plot([t0,tf],[pars['K'],pars['K']],color='k',linestyle='--',linewidth=3)
```

This code does not include the labeling commands. Nonetheless, as is apparent, simulations that begin with low concentrations end up converging to $x^{*,-} = 0$, and those that begin with high concentrations end up converging to $x^{*,+} \approx 92$. The meaning of low and high is relative to the point where positive production outpaces degradation, this happens for $x > K$ and $x < x^{*,+}$. The other key point is that dynamics converge and remain at one of the two equilibrium. Hence this positive feedback system is bistable. Note that the high-value equilibrium must satisfy $\frac{100x^4}{50^4+x^4} = x$, which is technically equal to $x^* = 91.9643....$

B. Visualizing state diagrams - towards bifurcation diagrams

What happens to the steady state dynamics given changes to parameters in a dynamical system? Will the equilibria change smoothly, or perhaps, quite rapidly? This last section explores this question by systematically varying the degree of positive feedback in the model, i.e., by changing β. Formal bifurcation theory involves more steps. Nonetheless, the flavor of how to generate a bifurcation diagram can be captured in the following two figures:

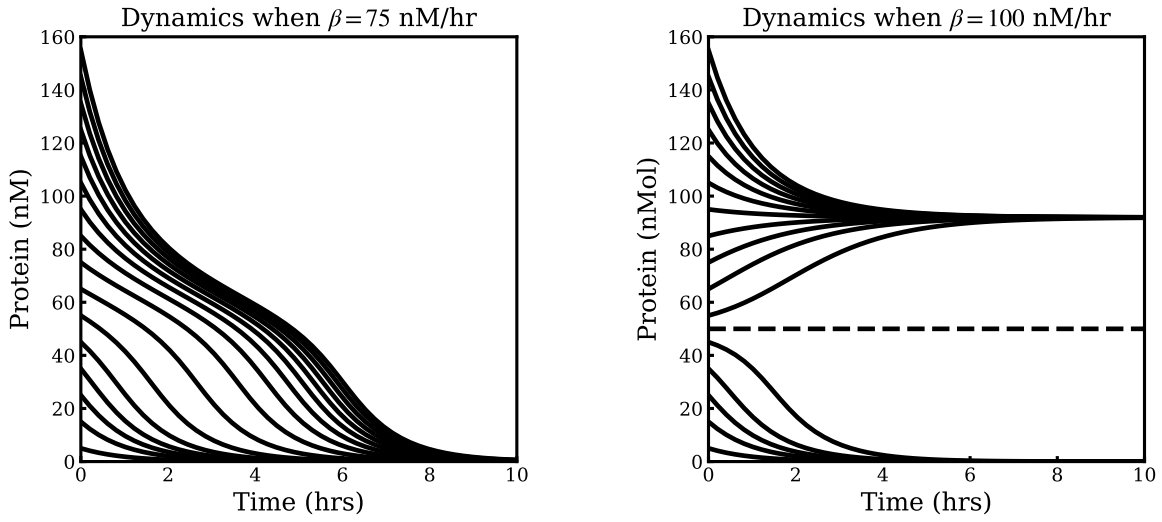




These two figures depict the production and degradation rates as a function of the protein concentration. In both cases $\alpha = 1$ nM/hr, $K = 50$ nM, and $n = 4$. However, the two cases differ in their maximum gene expression rate: $\beta = 85$ nM/hr and $\beta = 95$ nM/hr in the left and right panel respectively. This relatively small difference (approximately 10% of the total rate) leads to a significant change in output. When $\beta = 85$, then the degradation rate exceeds the production rate for all values of x, i.e., even with positive feedback. Hence, the only potential equilibrium should be when the curves intersect at a nullpoint rather than a nullcline. This equilibrium denotes the "Off" state when $x^* = 0$ as seen in the closed gray circle. by building upon the code written in the last challenge problem. In contrast when $\beta = 95$ there are 3 intersections. Each of these represents an equilibrium. However, only 2 of the three are stable (the "Off" and "On" states denoted by closed gray circles) and one is unstable (the intermediate state, denoted by an open circle). This analysis of the dynamical system suggests that small changes in a governing parameter can lead to qualitative changes in the output of the system – a bifurcation!

This analysis helps motivate the following framework for thinking about the relationship between dynamical systems and governing parameters. In some sense, one can think of the long-time behavior of a dynamical system as a map from $(\beta, K, n, \alpha) \rightarrow \{x^*\}$ where $\{x^*\}$ denote a set of fixed points – both stable and unstable. A bifurcation diagram visualizes how this map changes given variation in one (or more) of the input variables. There can be exceptions, e.g., when the steady state does not converge to a fixed point, but instead to some form of orbit. Yet, even in that case, the concept of a map holds, albeit to a higher dimensional space. This may seem abstract, so let's put these ideas into practice.

To start, modify the code from the previous challenge so that $\beta = 75$, $K = 50$ nM, $n = 4$ and $\alpha = 1$ /hr. Using the same code and all other parameters, you should find the following result, where the prior result is included for comparison:



Despite the positive feedback, all trajectories when $\beta = 75$ converge to the 'Off' state, where $x^* = 0$. This difference suggests that the intuition from comparison of the derivatives of the system was correct: a bifurcation in the system dynamics has happened somewhere between $\beta = 75$ and $\beta = 100$.

The homework problems will present a new challenge: to analyze and visualize bifurcations systematically in the context of a synthetic toggle switch – taking these ideas into a systems biology context.